

OOP Concepts — Cinema Ticket Booking System

OOP Concept	Where Used in Cinema Ticket System	Simple Example	Purpose
Encapsulation	Booking , ShowSeat	Keep status and totalAmount private and modify them through methods like confirm() , cancel() , and book() .	Protect data and control access
Abstraction	Payment	Payment defines pay() without showing how UPI, Card, or Cash payment works internally.	Hide implementation details
Inheritance	Payment hierarchy	UpiPayment , CardPayment , and CashPayment inherit from Payment .	Reuse common structure and behavior
Runtime Polymorphism	Payment processing	Payment* p = new UpiPayment(); p->pay(amount); calls the appropriate payment method at runtime.	Same interface, different behavior
Compile-Time Polymorphism	Methods or constructors	Use overloaded methods or constructors with different parameters.	Same name, different parameter lists
Static Members	Booking	static int nextBookingId is shared by all booking objects to generate unique IDs.	Maintain shared class-level data
this Keyword	Constructors and methods	this->bookingId = bookingId; refers to the current object's member variable.	Refer to the current object
Composition	Cinema → Screen → Seat	Cinema contains Screen objects and Screen contains Seat objects.	Represent strong ownership
Aggregation	Show → Movie , Booking → ShowSeat	Show stores a reference to an existing Movie , and	Represent weak ownership

OOP Concept	Where Used in Cinema Ticket System	Simple Example	Purpose
		Booking stores references to ShowSeat objects.	
Association	CinemaMainMenu → BookingService	CinemaMainMenu uses BookingService to perform booking and cancellation operations.	Allow objects to communicate

Most Important Examples to Remember

OOP Concept	Remember This
Encapsulation	Protect data using private members and controlled methods
Abstraction	Hide implementation details using Payment
Inheritance	Parent → Child , e.g. Payment → UpiPayment
Runtime Polymorphism	Same parent interface → different child behavior
Compile-Time Polymorphism	Same method name → different parameters
Static Members	One shared variable for all objects
this Keyword	Refers to the current object
Composition	Strong HAS-A relationship , e.g. Cinema → Screen
Aggregation	Weak HAS-A relationship , e.g. Show → Movie
Association	One class uses another class , e.g. Menu → BookingService